



A Python library for Trgen Device

TrGEN Device is the ultimate platform for scientific experiments with the easiest API ever.

pypi v1.1.4 python 3.7 | 3.8 | 3.9 license MIT

The full documentation is available [here](#).

Features

- **Nanosecond time resolution** --- every trgenpy features is tested with signals that perform into the nanosecond
- **Easy coding** --- code is clean and well documented for a simple use.
- **Open Source** --- feel free to read and spread the code, do your forks and share it!
- **Customize** --- with carefully-designed sidebar navigation and inter-page links.
- **Configuration Export/import** --- your TrGEN session can be imported and exported on file where you want.

Index

-  **Getting started**
 - [How to install](#)
 - [Import the library](#)
 - [Client](#)
-  **How to use**
 - [Sending Default Single Trigger to BNCO](#)
 - [Send Trigger\(s\) to Multiple Ports](#)
 - [Sending Markers](#)
 - [Achieving maximum temporal precision \(<10 ms\)](#)
-  **Programming Custom Trigger Signals**
 - [TrgenPin](#)
 - [TrgenPort](#)
 - [Custom Instruction Set](#)
 - [Concatenate Instructions](#)
 - [Instruction Helpers](#)
 - [Write Instruction on TRGenPort](#)
 - [Event Listening](#)
 - [Listen for Default Simple to BNCO](#)
 - [Listen for Markers](#)
 - [Listen for Custom Triggers](#)
 - [Set and listen to the photodiode](#)
-  **Native Commands**
 - [Program TrgenPort](#)

- Start Trigger
 - Set GPIO I/O Direction
 - Request Implementation
 - Request TrgenPort Status
 - Set The Trigger Level
 - Get The Trigger Level
 - Get the GPIO I/O Direction
 - Stop the TrgenPort
 -  **Helpers**
 - Reset TrgenPort
 - Get Default Duration
 - Set Default Duration
 - Get Implementation
 -  **Export/Import configuration**
 - Configuration Manager
 - Create Configuration
 - Export Configuration
 - Import Configuration
 - Configuration file format
 -  **E-Prime integration**
 -  **Examples**
-

Getting started

How to install

To install this package use is required **Python >= 3.9**

Install the TrGEN python library by opening the Command Prompt (cmd) and type:

```
pip install trgenpy
```

Once installed, open a code editor that allows you to write Python scripts, such as Visual Studio Code.

Import the library in the code editor

```
# import tha library like this:
import trgenpy as tp
# instantiate the object to enable communication with the TrGEN Device (the
client)
# like this:
client = tp.TrgenClient()
```

All functions in the trgenpy library should be called with the **tp.** prefix (e.g., **tp.function_name**)

Client

The **TrgenClient** object stores all the information about the socket connection between the user PC and the TrGEN Device.

Once the **client** object is created, connection between user PC and TrGEN Device is achieved with:

```
client.connect()
```

It is also possible to check the availability of the device (useful to check if the TrGEN Device is properly connected)

```
isAavailable = client.is_available() # returns true or false
```

How to use

Using TrGEN device, it is possible to send triggers via parallel port or send/receive triggers via BNC and GPIO (General Purpose Input Output).

The user may choose to send individual triggers or markers via parallel ports' **PINs**.

With TrGEN, it is also possible to send **parallel multiple triggers** via different parallel ports and BNC/GPIO ports. The triggers will be sent together with nanosecond precision.

Sending Default Single Trigger to BNCO

To send a default trigger (positive square wave with duration of 20 μ s) to the BNCO the **sendTrigger()** (without input argument) function can be used:

```
client.sendTrigger()
```

Send Trigger(s) to Multiple Ports

An input argument into **sendTrigger()** will send a trigger through the specified **PIN**

```
client.sendTrigger([tp.TrgenPin.NS0])
```

In this way, the TrGEN device will send a default trigger from **PIN 0**.

TrGEN **PINs** legend is illustrated in the TrGEN manual.

Note that the input argument of the **sendTrigger()** function must be a **list with square brackets**

To send multiple triggers at the same time (i.e., with nanosecond time resolution), multiple arguments can be specified in the `sendTrigger()` function as a list of type `TrgenPin`

```
client.sendTrigger([tp.TrgenPin.NS0,tp.TrgenPin.GPIO0])
```

This will send two triggers from the **PIN 0** of the NS parallel port and from the **PIN 0** of the GPIO at the same time.

This function can also be used to test whether signals on PINs are properly functioning. See the `TrgenPin` documentation.

Sending Markers

To send a marker to bioamplifiers equipped with parallel ports used for triggering and registering events, the `sendMarker()` function may be used with the following arguments like this:

```
client.sendMarker(markerNS=12,markerSA=139,markerGPIO=42)
```

NOTE: The input identifiers (`markerNS`, `markerSA`, `markerGPIO`) are **case-sensitive** and must not be misspelled.

The `sendMarker()` function accepts every combination of inputs in any order the user wish.

Another example is provided below:

```
client.sendMarker(markerSA=20,markerNS=9)
```

In this case the TrGEN device will send **two markers** from the SA and NS ports (20 and 9 respectively), while no markers will be sent from the GPIO port.

`sendMarker(markerNS=i,markerSA=j,markerGPIO=k)` will send the appropriate triggers to generate the desired markers `i`, `j`, `k` on the electrophysiological signal from ALL the equipped output ports (NS, SA, GPIO) on the TRGen device.

Achieving maximum temporal precision (<10 ms)

The TrGEN device can reach higher temporal resolution by programming the triggers or markers in advance, since both the `client.sendTrigger()` and `client.sendMarker()` functions require some time to prepare the instructions that will be sent to the device to emit a trigger or marker.

To prepare a trigger or marker in advance, just set the `autoStart` argument to `False` as the following example:

```
client.sendTrigger([tp.TrgenPin.NS0],autoStart=False) # This line will program the trigger in advance
```

```
## OR
client.sendMarker(markerSA=20,markerNS=9,autoStart=False) # This line will program
the markers in advance

# --- Some lines of your script ---

client.start() # This line will immediatly send out the trigger/marker to TrGEN
device
```

NOTE: The `autoStart` argument is set to `True` by default. The time needed to program triggers or markers instructions depends on the hardware and current computational load of the PC running trgenpy. Programming triggers or markers instructions may take up to hundreds of milliseconds. By programming triggers or markers in advance, it is possible to reach a temporal resolution below 10 milliseconds (dictated by the PC system). Remember that, when setting the `autostart` argument to `True` you will need to manually start the trigger(s) using the `client.start()` function.

Least Significant Bit (LSB)

If the marker number sent does not match with the one observed on the physiological signal, this is may due to an inverted mapping of the bioamplifier's `PINOUT`. To fix the issue, also it is possible to invert the bit order, just flag the `LSB` argument to `True` (`LSB` is `False` by default)

```
client.sendMarker(markerNS=13,LSB=True)
```

Auto-Stop

If set to `False`, the TRGen Device will continue the execution.

```
client.sendMarker(markerSA=100,LSB=True,stop=False)
# it's the same of calling client.stop() right after.
```

Programming Custom Trigger Signals

trgenpy offers some advanced functions and features for the user's needs to customize the output/input signal's behaviour.

TrgenPin

Each Pin on the TRGen Device has an unique `PIN_ID`. This is the classification of each Port grouped by connector Type

This is the complete list of the TRGen Device's pinout:

- NS IDs The NS pinout (only for used pins) goes from 0 to 7
 - `[NS0,NS1,NS2,NS3,NS4,NS5,NS6,NS7]`
- SA IDs The SA pinout (only for used pins) goes from 0 to 7

- [SA0,SA1,SA2,SA3,SA4,SA5,SA6,SA7]
- BNC I/O IDs
 - BNCO
 - BNCI
- GPIO IDs The GPIO pinout goes from 0 to 7
 - [GPIO0,GPIO1,GPIO2,GPIO3,GPIO4,GPIO5,GPIO6,GPIO7]

The user can assign a specific **PIN ID** to a variable by using the `TrgenPin` class.

The command may be in this form `TrgenPin.PIN_ID` and executed like this:

```
# defining pin
NS_fourth_pin = tp.TrgenPin.NS3 #Remember that the first pin is always 0!
```

For **PIN IDs** have a look at the `TrgenPin` documentation.

TrgenPort

The `TrgenPort` object defines a single trigger behaviour through its id. This is the list of supported `TrgenPin` values for the TRGrGen:

Instantiate the `TrgenPort` object with:

```
# a NeuroScan port is defined passing
# the TrgenPin.NS3 enum in the constructor argument
client = TrgenClient()
NS_port = client.createTrgenPort(tp.TrgenPin.NS3)

# but it's also possible to store the pin variable
BNCO_pin = TrgenPin.BNCO
BNCO_port = client.createTrgenPort(bnco_pin)
```

Custom Instruction Set

The `TrgenPort` object has a `memory` property. `memory` has to be filled with a list of `2^mtm1` slots, default cardinality is 32. Each slot can contain a single instruction that will be executed in order from the first to the last.

see the "[Concatenate Instructions](#)" section for further informations.

The supported instruction set for TRGen Device is:

- **Unactive For N µs**

The `TrgenPort` stays *unactive* for N µs

- **Active For N µs**

The TrgenPort stays *active* for $N \mu s$

- **Wait Positive Edge**

The TrgenPort waits until there is a positive edge

- **Wait Negative Edge**

The TrgenPort waits until there is a negative edge

- **Repeat from N, for X times**

The TrgenPort repeats the instruction sequence from the memory index N to the current one, for X times

- **End**

The TrgenPort ends its behaviour

- **Not Admissible**

This instruction is ignored, but it is *necessary* to fill the empty instruction list. **NOTE: if the space is empty, it will be filled automatically by trgenpy**

Concatenate Instructions

Each **TrgenPort** can be programmed with a bunch of instructions. the function `setInstruction()` can be use to store instructions into memory slots.

Instructions can be added in memory slots using the static function `setInstruction(memory_index, instruction)`, that adds a specific instruction in the desired memory index.

Index position is important since the program counter of TRGen Device execute each TrgenPort instructions from 0 to 2^{mtml} .

Remember, the `setInstruction(memory_index, instruction)` function **does not program** the **TrgenPort**, it just stores the instructon, that will be written using `writeTrgenMemory()`.

Instruction Helpers

Since for TRGen Device all instructions are "**bitmask** chunks" this library offers some helper functions that do the bitmask encoding. They can be used to define easily any instruction:

Instruction	1° Param	2° Param	Description
<code>istrUnactiveFor(us)</code>	μ seconds duration		Set the unactivation time for μ seconds
<code>istrActiveFor(us)</code>	μ seconds duration		Set the activation time for μ seconds

Instruction	1° Param	2° Param	Description
<code>istrWaitPE(tr)</code>	TrgenPin		Wait the positive edge for a specific TrgenPin
<code>istrWaitNE(tr)</code>	TrgenPin		Wait the negative edge for a specific TrgenPin
<code>istrRepeat(addr,time)</code>	Instruction address	Time of istrRepeat	Set the activation time for μ seconds
<code>istrEnd()</code>			End the behaviour
<code>istrNotAdmissible()</code>			Empty instruction, it has to be placed after the <code>istrEnd()</code>

Write Instruction on TRGenPort

Once the desired instruction set has be setted, the `memory` can be written using the function `writeTrgenMemory(trgenport=TrgenPin.BNCO)` to a specific `TrgenPort` by its `TrgenPin`. After this, the program can be executed invoking the static method `start()`

The following script use the whole instruction set in order to program a single `TrgenPort`

custom instructions examples

```

client = TrgenClient()
bnco = client.createTrgenPort(TrgenPin.BNCO)
impl = client.getImplementation()

# helper function used to clear the current TrgenPort memory
# it program the desired TrgenPort with a blank program
# made of one istrEnd() and all istrNotAdmissible() instructions
client.resetTrgenPort(bnco) # it is explained in the "Helpers" paragraph

bnco.setInstruction(0, istrActiveFor(5))
bnco.setInstruction(1, istrUnactiveFor(3))
bnco.setInstruction(2, istrRepeat(0,2))
bnco.setInstruction(3, istrWaitPE(TrgenPin.NS4))
bnco.setInstruction(4, istrUnactiveFor(3))
bnco.setInstruction(5, istrWaitNE(TrgenPin.NS0))
bnco.setInstruction(6, istrEnd()) # istrEnd is mandatory!
for i in range(7,impl.memory_length): # from 7 to 31!
    bnco.setInstruction(i, istrNotAdmissible()) # this also is mandatory

# write the program
# every instruction in bnco.memory
# is flashed into TRGen Device
client.writeTrgenMemory(bnco)

```

```
# execute the TRGen Device
client.start()
```

`writeTrgenMemory()` write only specific `TrgenPort`. This means that many custom programs can be written

```
bnco = client.createTrgenPort(TrgenPin.BNCO)
ns7 = client.createTrgenPort(TrgenPin.NS7)

# you can also use this helper in order to reset All TrgenPort
client.resetAllTrgenPorts()

bnco.setInstruction(0, istrActiveFor(5))
bnco.setInstruction(1, istrUnactiveFor(3))
bnco.setInstruction(2, istrEnd())
for i in range(3,31):
    bnco.setInstruction(i, istrNotAdmissible())

ns7.setInstruction(0, istrWaitPE(TrgenPin.BNCO))
ns7.setInstruction(1, istrActiveFor(5))
ns7.setInstruction(2, istrUnactiveFor(3))
ns7.setInstruction(3, istrEnd())
for i in range(4,31):
    ns7.setInstruction(i, istrNotAdmissible())

client.writeTrgenMemory(ns7)
client.writeTrgenMemory(bnco)

# execute the TRGen Device
client.start()
```

Event Listening

These are functions that helps on programming simple and customized behaviour on a simple trigger event on a specified `inputPin`.

Listen for Default Simple to BNCO

Configures the BNC output to automatically respond to signals received on a custom `inputPin`

```
from trgenpy import TrgenClient

client = TrgenClient()
client.connect()

# Respond to positive edge (default)
# default input inputPin is BNCI
# send default Trigger to BNCO
client.callbackCustomTrigger(inputPin=TrgenPin.BNCI)
```

```
# Program a behaviour that send default trigger when BNCI (default) is on negative edge
client.callbackCustomTrigger(ne=True,inputPin=TrgenPin.BNCI)

# Program a behaviour that send default trigger when NS6 is on negative edge
client.callbackCustomTrigger(ne=True,inputPin=TrgenPin.NS6)

# Program a behaviour that send default trigger when GPIO2 is on positive edge (default True)
# it automatically set the global GPIO direction to "input"
client.callbackCustomTrigger(ne=True,inputPin=TrgenPin.GPIO2)

# Remember to call .start(), because this is just Port programming
# Start listening
client.start()
```

Every time an input trigger is sent to the designed `inputPin`, the `callbackCustomTrigger` function will send a trigger signal to the BNC output port.

Listen for Input Triggers

Configures the GPIO connector (in "input direction" mode) to respond to signals received on a specific GPIO pin.

Set GPIO I/O Direction

You can use this helper to build the complete or partial pinout map to set.

```
# we choose to set just GPIO0 -> High and the GPIO1 -> Low
mask = DirectionConfig().out(TrgenPin.GPIO0).in(TrgenPin.GPIO1).build()

client.set_gpio_direction(mask)
```

You can also send a marker right after the input trigger has been received using the `callbackMarker` function, like this:

```
# Send a marker from NS port when GPIO2 is on negative edge
# Also set the Global GPIO direction to "input"
client.callbackMarker(ne=True, markerNS=50,inputPin=TrgenPin.GPIO2)

# Start listening
client.start()
```

You can also send multiple markers at once:

```
client.callbackMarker(ne=True, markerNS=50,markerSA=13,inputPin=TrgenPin.GPIO2)
```

NOTE

The TrGEN device can't wait for input triggers while sending others in output (i.e., wait for input trigger at GPIO2 and send a marker through markerNS=50). To do so you should create your script similar to this example:

```
# create a callbackMarker to wait for input triggers to GPIO2
client.callbackMarker(markerSA=70,inputPin=tp.TrgenPin.GPIO2)
# send a marker through SA port and RESETS the callbackMarker instruction
client.sendMarker(markerSA=70,stop=False)
# re-create the callbackMarker to keep waiting for input trigger to GPIO2
client.callbackMarker(markerSA=70,inputPin=tp.TrgenPin.GPIO2)
```

It's not allowed to send GPIO markers when the `inputPin` is GPIO.

```
# THIS WILL GENERATE ERRORS
client.callbackMarker(ne=True, markerGPIO=13,inputPin=tp.TrgenPin.GPIO2)

# GPIO cannot be used as both input and output simultaneously
```

Listen for Custom Triggers

Configures a custom output pin list to respond to a specific `inputPin` of type `TrgenPin`

```
# Program a default trigger to Neuroscan 7 and Synamps 8 pins on a negative front
edge from the BNCI port
client.callbackCustomTrigger(ne=True, trgenPinList=
[tp.TrgenPin.NS7,tp.TrgenPin.SA8], inputPin=tp.TrgenPin.BNCI)

# Start listening
client.start()

# Advanced configuration: use custom instruction set
# Configuration with custom instructions
instruction_set = [
    istrWaitPE(TrgenPin.BNCI),      # Wait for positive edge on BNCI
    istrActiveFor(50),              # Active for 50µs
    istrUnactiveFor(10),            # Inactive for 10µs
    istrActiveFor(30),              # Active for 30µs
    istrUnactiveFor(20),            # Inactive for 20µs
    istrActiveFor(20),              # Active for 20µs
    istrUnactiveFor(30),            # Inactive for 30µs
```

```

    istrEnd()                                # End program
]

# Program a custom trigger on GPIO8 and BNCO on a positive edge front (default
when not specified) with a custom instruction set
client.callbackCustomTrigger(trgenPinList=[TrgenPin.GPIO8,TrgenPin.BNCO],
inputPin=TrgenPin.BNCI,customInstructions=instruction_set)

# Start listening
client.start()

```

NOTE

Programming custom trigger does not include the automatic start. It's always recommended to invoke the `start()` function once your custom instruction set is written

Set and listen to the photodiode

The photodiode can be used to trigger events when a target luminance overcomes a defined threshold. To set up the photodiode, first it must be calibrated with a defined luminance. Then the user may set a luminance threshold to enable trigger(s) or marker(s) on output ports:

```

client.startCalibration(0) # The photodiode will start the calibration and will
detect the luminance in 100 ms.
                        # The function argument is always zero

thr = client.getThresholdValue(0) # The calibrated threshold will be stored in
the 'thr' variable.
                        # The function argument is always zero

print(f"The calibrated threshold is:{thr}") # Display the calibrated threshold

client.setThresholdValue(0,500) # Set the target threshold (500 in this example).
                        # When it is overcome, TrGEN will emit a
trigger/marker.
                        # The first function argument is always zero

#---Your lines of code here---

client.callbackMarker(markerNS=100,inputPin=TrgenPin.PD,ne=True) # It is
possible to use the client.callbackTrigger function too.
                                                                # TrgenPin.PD
is the photodiode input PIN.
                                                                # ne=True is a
mandatory argument when the photodiode is the inputPin.
client.start() #Starts listening

```

NOTE: The photodiode response to the target threshold luminance is instantaneous and the output trigger(s)/marker(s) programmed by the `client.callbackTrigger` and `client.callbackMarker` will be

sent from the TrGEN device with nanosecond time resolution.

Native Commands

TRGen devices can receive some commands, the full instruction set includes:

- Program TrgenPort
- Start TrgenPort
- Set GPIO I/O Direction
- Request Implementation parameters
- Request TrgenPort Status
- Set the TrgenPort level
- Get the TrgenPort Level
- Get GPIO I/O Direction
- Stop the TrgenPort

Program TrgenPort

You can program a trigger behaviour in two ways:

1. **Default trigger** using `programDefaultTrigger()`

```
tr = client.createTrgenPort(TrgenPin.GPIO00)
# This sends an impulse of duration (default 20µs)
client.programDefaultTrigger(tr) # impulse of default 20µs
```

```
tr = client.createTrgenPort(TrgenPin.GPIO00)
# This override the standard duration and sends an impulse.
client.programDefaultTrigger(tr, us=50) # impulse of 50µs

# In this way, all the successive triggers
# sent from GPIO00 PIN will have a duration of 50 microseconds
```

2. **Advanced programming** by manually setting instructions with `setInstruction()`

```
tr = client.createTrgenPort(TrgenPin.NS1)
tr.setInstruction(0, istrActiveFor(10))
tr.setInstruction(1, istrUnactiveFor(5))
tr.setInstruction(2, istrRepeat(0, 3))
tr.setInstruction(3, istrEnd())
client.writeTrgenMemory(tr)
```

Start Trigger

After programming triggers, you can start them with:

```
client.start()
```

This command makes the Trgen execute all programmed triggers.

Remember: you must have previously written at least one **TrgenPort** program into its **memory**. **NOTE:** You don't need to call the `client.start()` function when calling `sendMarker`, `sendTrigger` and `sendCustomTrigger`, as it is already integrated.

Request Implementation

You can get information about the hardware configuration (number of channels, memory length, etc.) with:

```
impl = client.getImplementation()
print(impl.memory_length) # max number of instructions per trigger
```

Request TrgenPort Status

To check current status of triggers (active/inactive state):

```
status = client.get_status()
print(status)
```

This returns an integer bitmask where each bit represents the state of a trigger pin.

Set the Trigger level

It is possible to change the polarity (active-high or active-low) for any **TrgenPort**: Use the helper function `set_level` like this in order to build the complete or partial pinout map to set.

```
# we choose to set just GPIO0 -> High and the GPIO1 -> Low
mask = LevelConfig().high(TrgenPin.GPIO0,
TrgenPin.BNCO).low(TrgenPin.GPIO1).build()
# once you have it, you can use it as argument in the set_level function
client.set_level(mask)
```

Get the Trigger Level

To check trigger polarity:

```
level = client.get_level()
print(bin(level))
```

Get the GPIO I/O Direction

To retrieve the current GPIO configuration:

```
gpio_state = client.get_gpio_direction()
print(bin(gpio_state))
```

Stop the TrgenPort

To stop the TRGen device use the `stop()` function

```
gpio_state = client.stop()
```

Helpers

Here's some utility functions used to simplify the TRGen Device interaction

Reset TrgenPort

In order to reset a specific `TrgenPort` program, the following static functions can be used:

- `resetTrgenPort(trgenport)`
- `resetAllTrgenPorts()`

```
bnco = TrgenPort(TrgenPin.BNCO)
client.resetTrgenPort(bnco) # reset just bnco
# it basically does
# bnco.setInstruction(0, istrEnd())
# for i in range(1, 31):
#     bnco.setInstruction(i, istrNotAdmissible())

client.resetAllTrgenPorts() # reset all TrgenPorts
```

Set Default Duration

In [this](#) and [this](#) section is explained how to send a default trigger to BNCO or multiple TrgenPorts

Default duration is 20µs, but it can be changed using `setDefaultDuration(duration_us)` like this:

```
client.setDefaultDuration(100)
client.sendTrigger() # now this default trigger lasts 100µs
```

Get Default Duration

To get standard new duration use `getDefaultDuration()` like this:

```
new_duration = client.getDefaultDuration() # returns an int value
```

Get Implementation

The TRGenDevice has its own "settings" stored into the `impl` object defined by class `TrgenImplementation`. `impl` is a `TrgenClient` property and can be reached invoking the static function `getImplementation()` like this:

```
client = TrgenClient()
impl = client.getImplementation()
```

`TrgenImplementation` contains informations about:

- `TRGen cardinality`
- `mtml`
- `memory_length`

TRGen Cardinality

Its a bunch of properties describing the maximum number of TrgenPort by type:

- `ns_num` (8)
- `sa_num` (8)
- `bnco_num` (1)
- `bnci_num` (1)
- `gpio_num` (8)

Max TRGen Memory Length (`mtml`)

Each TrgenPort can support a list of N instructions, where $N = 2^{MTML}$ (Max TrgenPort Memory Length). the Memory Length is a "magic number" hardcoded inside the triggerbox firmware. In the current versions is 5. You can access to the `mtml` value by

```
# Add this in try block to manage errors
try:
    impl = client.getImplementation()
    mtml = impl.mtml
except InvalidAckError as e:
    print(f"⚠️ ACK sbagliato: {e}")
except AckFormatError as e:
    print(f"⚠️ ACK malformato: {e}")
except TimeoutError as e:
    print(f"⌚ Timeout: {e}")
```

Memory Length

`memory_length` is the max number of instructions per `TrgenPort`, that is 2^{mtml} .

Export/Import configuration

You can export the current device configuration to a file and import it back when needed. This is useful for backup, sharing, or restoring device settings. You basically save the `TrgenClient` state.

Configuration Manager

The class `ConfigurationManager` offers function for the entire lifecycle of a configuration. creation, import and export

Create Configuration

You can create the current configuration to a JSON file using the `export_config()` method:

```
config = TrgenConfiguration()

# Set metadata
config.metadata.project_name = "Advanced EEG Experiment"
config.metadata.description = "Configuration with custom timing for multi-modal experiment"
config.metadata.author = "Research Team"

# Set custom default duration
config.defaults.default_trigger_duration_us = 50 # 50 microseconds
config.defaults.auto_reset_enabled = True
config.defaults.auto_reset_delay_us = 200

# Configure NS0 with custom instructions (100µs trigger)
ns0_config = TriggerPortConfig(32)
ns0_config.id = TrgenPin.NS0
ns0_config.name = "EEG Start Marker"
ns0_config.type = "NeuroScan"
ns0_config.enabled = True
ns0_config.custom_duration_us = 100
ns0_config.notes = "Used for experiment start markers"

# Program custom instructions for NS0
ns0_config.memory_instructions = [0] * 32
ns0_config.memory_instructions[0] = istrActiveFor(100) # Active for 100µs
ns0_config.memory_instructions[1] = istrUnactiveFor(500) # Inactive for 500µs
ns0_config.memory_instructions[2] = istrEnd() # End sequence
ns0_config.last_instruction_index = 2
ns0_config.programming_state = "Programmed"

config.trigger_ports["NS0"] = ns0_config

# Configure SA0 with different timing
sa0_config = TriggerPortConfig(32)
sa0_config.id = TrgenPin.SA0
```

```

sa0_config.name = "Stimulus Trigger"
sa0_config.type = "Synamps"
sa0_config.enabled = True
sa0_config.custom_duration_us = 200
sa0_config.notes = "Used for stimulus presentation markers"

# Program custom instructions for SA0
sa0_config.memory_instructions = [0] * 32
sa0_config.memory_instructions[0] = istrActiveFor(200) # Active for 200µs
sa0_config.memory_instructions[1] = istrEnd() # End sequence
sa0_config.last_instruction_index = 1
sa0_config.programming_state = "Programmed"

config.trigger_ports["SA0"] = sa0_config

# Configure BNCO for TMS applications
bnco_config = TriggerPortConfig(32)
bnco_config.id = TrgenPin.BNCO
bnco_config.name = "TMS Output Control"
bnco_config.type = "BNCO"
bnco_config.enabled = True
bnco_config.custom_duration_us = 1000 # 1ms for TMS
bnco_config.notes = "Controls TMS device output"

# Program TMS-specific instructions
bnco_config.memory_instructions = [0] * 32
bnco_config.memory_instructions[0] = istrActiveFor(1000) # Active for 1ms
bnco_config.memory_instructions[1] = istrUnactiveFor(5000) # Inactive for 5ms
# safety margin
bnco_config.memory_instructions[2] = istrEnd() # End sequence
bnco_config.last_instruction_index = 2
bnco_config.programming_state = "Programmed"

config.trigger_ports["BNCO"] = bnco_config

# Add some standard ports without custom programming (will use defaults)
for i in range(1, 4): # NS1, NS2, NS3
    port_config = TriggerPortConfig(32)
    port_config.id = getattr(TrgenPin, f'NS{i}')
    port_config.name = f"NeuroScan {i}"
    port_config.type = "NeuroScan"
    port_config.enabled = True
    config.trigger_ports[f"NS{i}"] = port_config

```

The exported configuration includes:

- All programmed TrgenPort instructions
- GPIO direction settings
- Trigger level configurations
- Default duration settings

Export Configuration

Export is easy:

```
# Export current client state (if you have an existing setup)
print("\n4. Exporting current client configuration...")
export_file = TrgenConfigurationManager.export_configuration(
    client, # is the current TrgenClient used
    "config_examples/current_setup.trgen", # destination path
    project_name="Current Device Setup", # project name
    description="Exported configuration from current client state", # description
    author="TrGEN User" # author
)
```

Import Configuration

To load a previously saved configuration, use the `import_config()` method:

```
from trgenpy import TrgenClient

client = TrgenClient()
client.connect()

# Save the configuration to file
config_file = "config_examples/advanced_experiment.trgen"
imported_config = TrgenConfigurationManager.import_configuration(client,
    config_file)

client.start()
```

Configuration File Format

The configuration is stored in JSON format with the following structure:

```
{
  "metadata": {
    "version": "1.0",
    "created_at": "2025-01-15T10:30:00.123456",
    "modified_at": "2025-01-15T10:30:00.123456",
    "description": "Configuration for EEG experiment with custom triggers",
    "project_name": "Multi-modal Experiment",
    "author": "Research Team"
  },
  "defaults": {
    "default_trigger_duration_us": 50,
    "default_log_level": "info",
    "default_timeout_ms": 2000,
    "auto_reset_enabled": true,
  }
}
```

```

        "auto_reset_delay_us": 200
    },
    "trigger_ports": {
        "NS0": {
            "id": 0,
            "name": "EEG Start Marker",
            "type": "NeuroScan",
            "enabled": true,
            "custom_duration_us": 100,
            "memory_instructions": [268435712, 536871424, 1, 0, 0, ...],
            "memory_length": 32,
            "last_instruction_index": 2,
            "programming_state": "programmed",
            "last_programmed_at": "2025-01-15T10:30:00",
            "notes": "Used for experiment start markers"
        },
        "BNCO": {
            "id": 16,
            "name": "TMS Output Control",
            "type": "BNCO",
            "enabled": true,
            "custom_duration_us": 1000,
            "memory_instructions": [805306880, 1342177792, 1, 0, 0, ...],
            "memory_length": 32,
            "last_instruction_index": 2,
            "programming_state": "programmed",
            "notes": "Controls TMS device output"
        }
    },
    "network": {
        "ip_address": "192.168.123.1",
        "port": 4242,
        "timeout_ms": 2000
    }
}

```

This feature is particularly useful for:

- Sharing configurations across different experiments
- Version control of trigger setups
- Quick restoration of complex configurations
- Backup of working setups

E-Prime integration

trgenpy offers a wrapper version to integrate its behaviour inside the [E-Prime](#) software via Python COM-visible.

Install pywin32

```
pip install pywin32
```

Register the COM wrapper

```
python trgen_com.py --register
```

So that you can use in E-Prime like this

```
Dim tb
Set tb = CreateObject("Trgen.COM")

If tb.IsAvailable() Then
    tb.Connect
    tb.SendTrigger
    tb.SendMarker 13
Else
    MsgBox "Trgen not available!"
End If
```

Examples

See the [examples](#) directory for the complete examples

Send Trigger BNCO

```
from trgenpy import
TrgenClient,TrgenPin,istrActiveFor,istrUnactiveFor,istrEnd,istrWaitNE,istrWaitPE

# create the Client for the Trgen
client = TrgenClient() # eventually TrgenClient(ip="192.168.123.2")

client.connect()
client.sendTrigger()
```

Send Trigger

```
from trgenpy import
TrgenClient,TrgenPin,istrActiveFor,istrUnactiveFor,istrEnd,istrWaitNE,istrWaitPE

# create the Client for the Trgen
client = TrgenClient() # eventually TrgenClient(ip="192.168.123.2")
```

```
client.connect()
client.sendTrigger([TrgenPin.NS0,TrgenPin.GPI00])
```

Custom Trigger

```
from trgenpy import
TrgenClient,TrgenPin,istrActiveFor,istrUnactiveFor,istrEnd,istrWaitNE,istrWaitPE
# create the Client for the Trgen
client = TrgenClient() # eventually TrgenClient(ip="192.168.123.2")

client.connect()

def set_bnco_up():
    # check if the device is online
    if client.is_available():
        print("Trgen is connected")

        # build a trigger
        bnco = client.createTrgenPort(TrgenPin.BNCO)
        bnco.setInstruction(0, istrActiveFor(5))
        bnco.setInstruction(1, istrUnactiveFor(3))
        bnco.setInstruction(2, istrEnd())
        # send the trigger
        client.writeTrgenMemory(bnco)

        # start the sequence
        client.start()

        # stop the sequence
        client.stop()

set_bnco_up()
```